Project - High Level Design
On
Project Title

Sports Planning Assistant Agent

Course Name: Agentic AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	Ayushi Parsai	EN22CS301257
2	Devansh Mittal	EN22CS301326
3	Ayush Kourav	EN22CS301248
4	Deepesh Vyas	EN22CS301319

Group Name:06D3

Project Number:AAI-30

Industry Mentor Name: Prof Suraj Nayak

University Mentor Name: Prof. Maya Yadav

Academic Year: 2025-26

Table of Contents

1. Introduction.
   1.1. Scope of the document.
   1.2. Intended Audience
   1.3. System overview.

2. System Design.
   2.1. Application Design
   2.2. Process Flow.
   2.3. Information Flow.
   2.4. Components Design
   2.5. Key Design Considerations
   2.6. API Catalogue.

3. Data Design.
   3.1. Data Model
   3.2. Data Access Mechanism
   3.3. Data Retention Policies
   3.4. Data Migration

4. Interfaces

5. State and Session Management

6. Caching

7. Non-Functional Requirements
   7.1. Security Aspects
   7.2. Performance Aspects

8. References

1.Introduction

The **Sports Planning Assistant Agent** is an intelligent system designed to assist users in planning, analyzing, and optimizing sports-related activities such as match strategies, player performance tracking, and event scheduling. The system leverages AI-driven agents to automate planning tasks, generate insights, and provide recommendations based on historical and real-time data.
This project aims to reduce manual effort in sports planning while improving decision-making through data-driven insights. It integrates multiple modules such as planning agents, performance analyzers, and strategy generators to deliver a comprehensive solution.

1.1 Scope of the Document

This document provides a **high-level architectural overview** of the Sports Planning Assistant Agent system. It covers system design, data flow, component structure, APIs, and non-functional requirements. The scope includes describing how different modules interact and how the system ensures scalability, security, and performance.

1.2 Intended Audience

This document is intended for:

- Software developers and engineers
- System architects
- Project stakeholders and managers
- QA and testing teams

It helps them understand the system architecture and design decisions.

1.3 System Overview

The system consists of multiple intelligent agents working collaboratively:

- **Planning Agent** – Handles scheduling and planning tasks
- **Performance Agent** – Analyzes player/team performance
- **Strategy Agent** – Suggests strategies based on data

The system uses APIs, databases, and AI models to process inputs and generate outputs in the form of reports and recommendations.

2. System Design

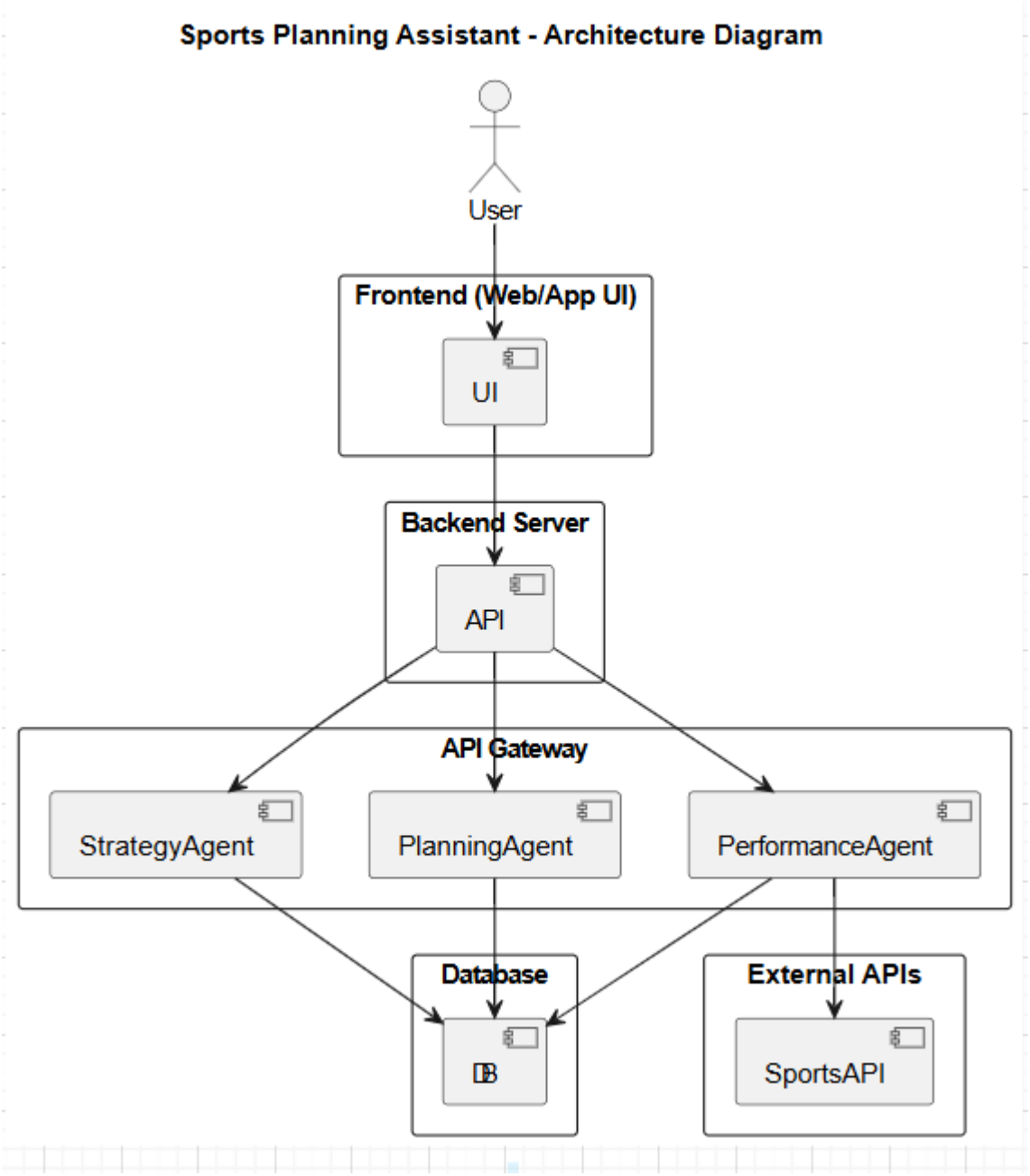
The system follows a **modular and scalable architecture**, ensuring flexibility and maintainability. It adopts a layered design approach consisting of presentation, business logic, and data layers.

2.1 Application Design

The application is designed using a **multi-tier architecture**:

- **Frontend Layer** – User interface for input and visualization
- **Backend Layer** – Handles business logic and agent coordination
- **AI/Agent Layer** – Processes data and generates insights
- **Database Layer** – Stores structured and unstructured data

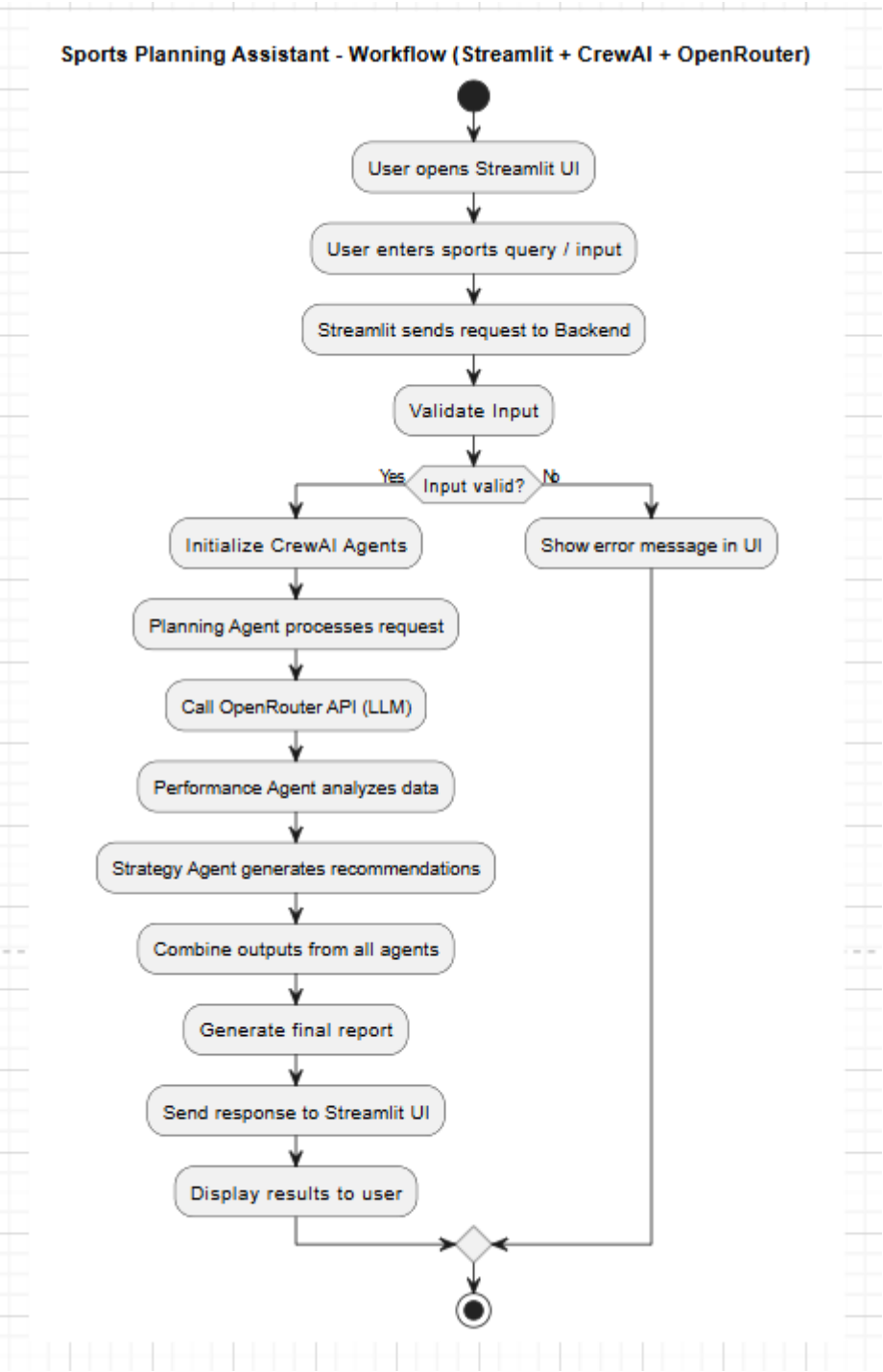
This separation improves scalability and maintainability.



2.2 Process Flow

The process begins when a user provides input such as match details or player data. The request is sent to the backend, where the planning agent processes it. The performance agent analyzes

relevant data, and the strategy agent generates recommendations. Finally, the system compiles results into a structured report and displays them to the user.

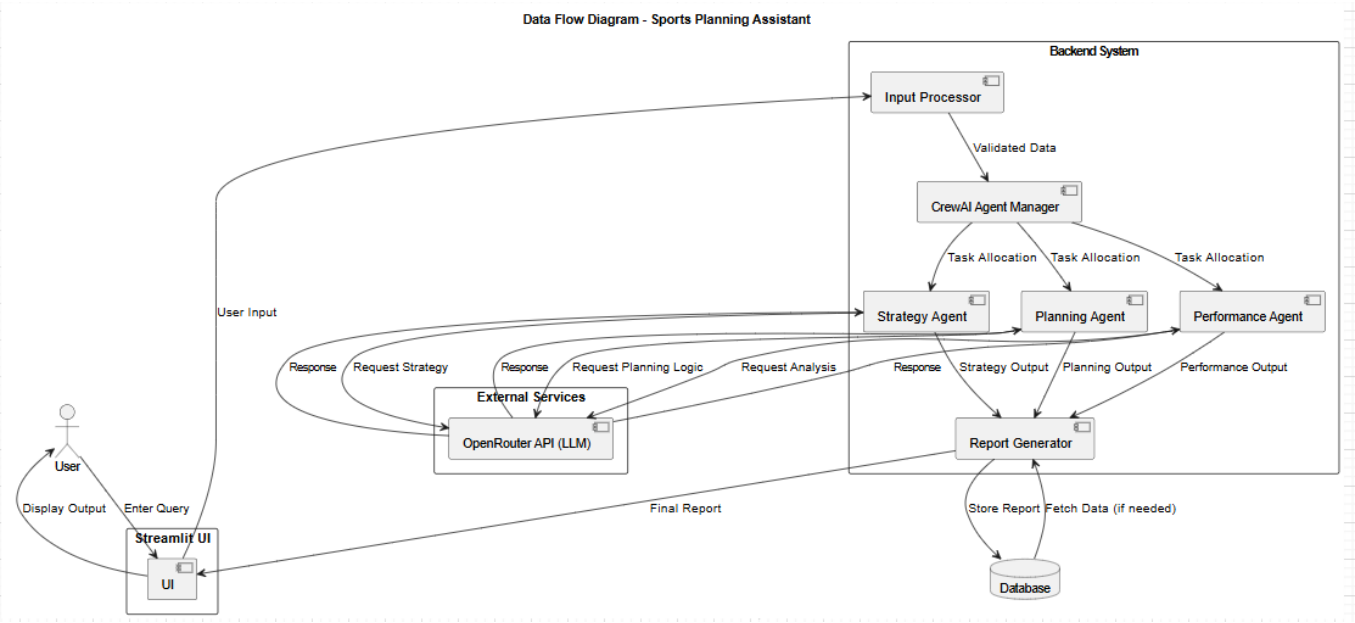


2.3 Information Flow

Data flows through the system in a structured manner:

- User input → API layer → Backend processing
- Backend → AI agents → Data analysis

AI agents → Response generation → Frontend display
This ensures smooth communication between components.

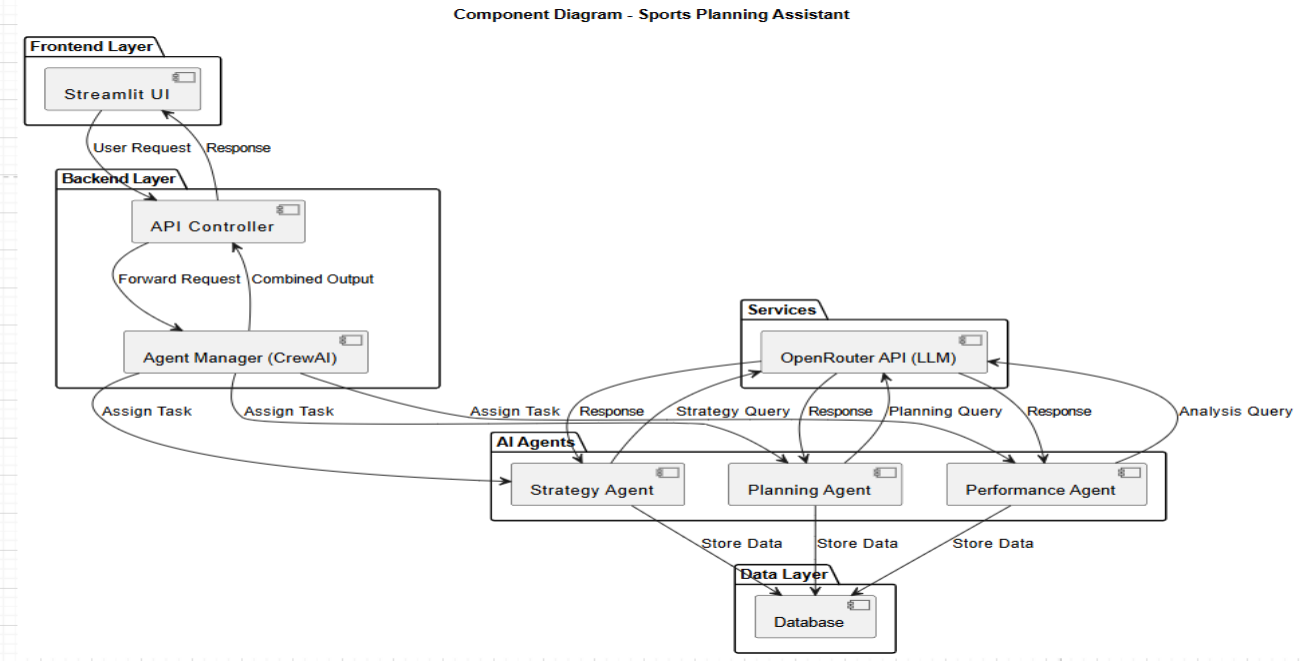


2.4 Components Design

The system consists of the following key components:

- **User Interface Module** – Handles user interaction
- **API Gateway** – Manages communication between frontend and backend
- **Agent Manager** – Coordinates different AI agents
- **Data Processing Module** – Handles analytics and computations
- **Database System** – Stores user data, match data, and reports

Each component is loosely coupled for better scalability.



2.5 Key Design Considerations

Important design considerations include:

- **Scalability** – System should handle increasing users and data
- **Reliability** – Ensure minimal downtime
- **Extensibility** – Easy to add new features or agents
- **Maintainability** – Modular code structure
- **Performance Optimization** – Efficient data processing

2.6 API Catalogue

The system exposes several APIs:

- **User API** – Handles user inputs and authentication
- **Planning API** – Generates schedules and plans
- **Performance API** – Fetches and analyzes performance data
- **Strategy API** – Provides strategy recommendations
- **Report API** – Generates downloadable reports

These APIs follow REST principles for easy integration.

3. Data Design

3.1 Data Model

The data model includes:

- **User Entity** – Stores user details
- **Match Entity** – Contains match information
- **Player Entity** – Stores player statistics
- **Performance Data** – Tracks historical performance
- **Reports Entity** – Stores generated outputs

Relationships are maintained using relational or NoSQL databases.

3.2 Data Access Mechanism

The system uses:

- ORM (Object Relational Mapping) for database interaction
- APIs for external data integration
- Query optimization techniques for fast data retrieval

3.3 Data Retention Policies

Data is stored based on usage:

- Temporary data is deleted after processing
- Historical data is retained for analysis
- Sensitive data is encrypted and stored securely

3.4 Data Migration

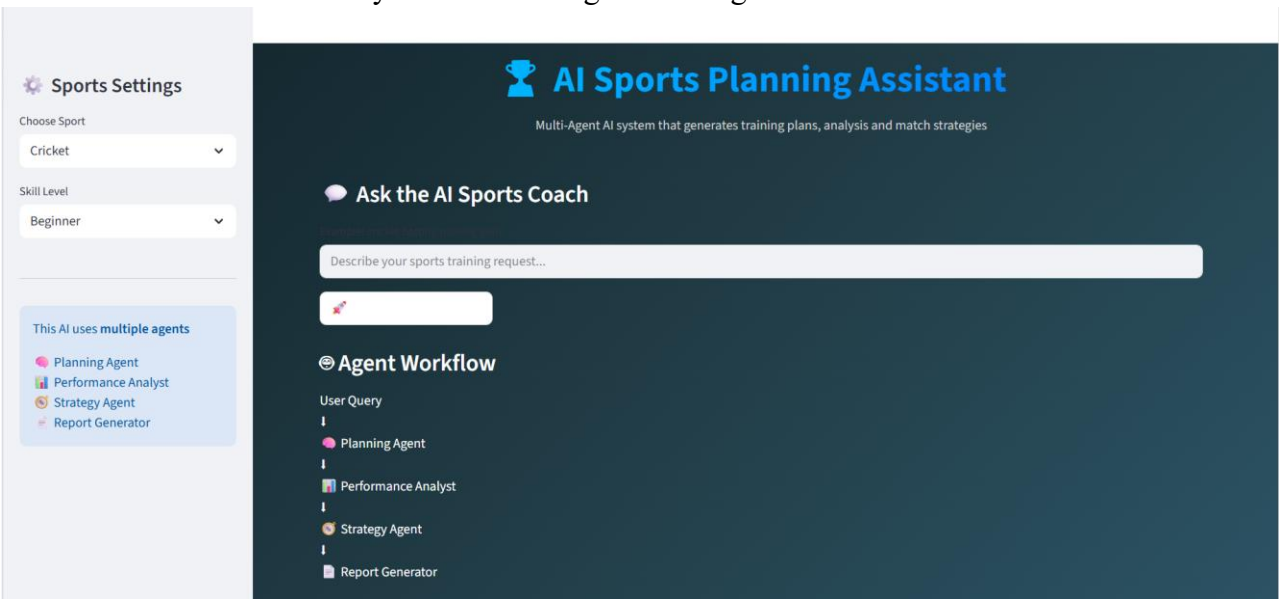
The system supports migration through:

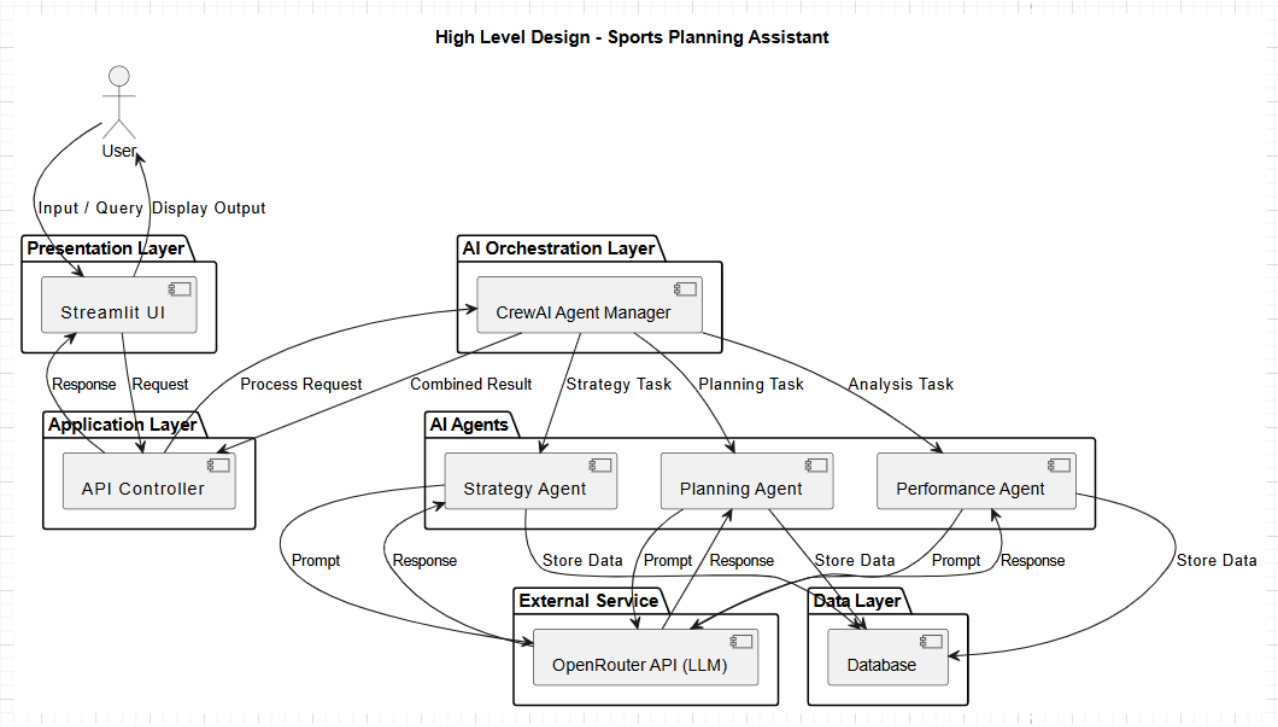
- Backup and restore mechanisms
- Version-controlled database updates
- Data transformation tools for compatibility

4. Interfaces

The system provides:

- **User Interface (UI)** – Web-based interface for interaction
- **API Interfaces** – For integration with external systems
- **Admin Interface** – For system monitoring and management





5. State and Session Management

The system maintains user sessions using:

- Token-based authentication (e.g., JWT)
- Session storage for temporary user data
- Stateless backend design for scalability

6. Caching

Caching is used to improve performance:

- Frequently accessed data is stored in cache
- Reduces database load
- Improves response time

Technologies like Redis can be used.

7. Non-Functional Requirements

7.1 Security Aspects

Security measures include:

- Authentication and authorization
- Data encryption
- Secure API communication (HTTPS)
- Protection against vulnerabilities like SQL injection

7.2 Performance Aspects

Performance is ensured through:

- Efficient algorithms

- Load balancing
- Caching mechanisms
- Scalable infrastructure

8. References

- System design best practices
- AI agent architecture concepts
- REST API design guidelines
- Database design principles